

# ML Project Report

## High-Performance Prediction Using Bagging

*Algorithm: Random Forest | Dataset: Telco Customer Churn*

**Submitted by:**

Aditya Roshan Singh

Course: AIML Mini Project | June 2025

# 1. Introduction

## 1.1 Dataset Description

The dataset used in this project is the IBM Telco Customer Churn dataset, sourced from Kaggle. It contains records of 7,043 telecom customers with 21 features describing customer demographics, subscribed services, account information, and whether the customer churned (left the service).

The target variable is 'Churn' — a binary classification label where:

- 0 = Customer did not churn (retained)
- 1 = Customer churned (left the service)

The dataset presents a class imbalance challenge: approximately 73.5% of customers are non-churners (class 0) and only 26.5% are churners (class 1). This is a realistic and important distribution for business problems, as churn is typically the minority event that companies most want to predict.

## 1.2 Choice of Algorithm — Random Forest

Random Forest was selected as the algorithm for this project for the following reasons:

- It is a Bagging (Bootstrap Aggregating) ensemble method that builds multiple independent decision trees in parallel and combines their outputs via majority voting.
- It handles mixed data types (numerical and categorical) well and is robust to outliers.
- It naturally provides feature importance scores, making model interpretation straightforward.
- It is less prone to overfitting compared to a single decision tree due to the averaging effect of multiple trees.
- It works well on moderately sized tabular datasets like the Telco Churn dataset without requiring extensive feature engineering.

The core principle: Random Forest builds  $N$  decision trees, each trained on a random bootstrap sample of the data and using a random subset of features at each split. The final prediction is determined by majority vote across all trees.

## 2. Exploratory Data Analysis & Visualizations

### 2.1 Box Plot — MonthlyCharges Distribution

The box plot below visualizes the spread of the MonthlyCharges feature, which was identified as one of the top predictors of churn. Key observations:

- The median monthly charge is approximately \$70.
- The interquartile range (IQR) spans from roughly \$35 to \$90, indicating a wide spread in pricing.
- No extreme outliers are visible — the whiskers extend to the minimum (~\$18) and maximum (~\$119) without individual outlier points.
- The symmetric distribution suggests that the MonthlyCharges feature does not require skewness correction before modeling.

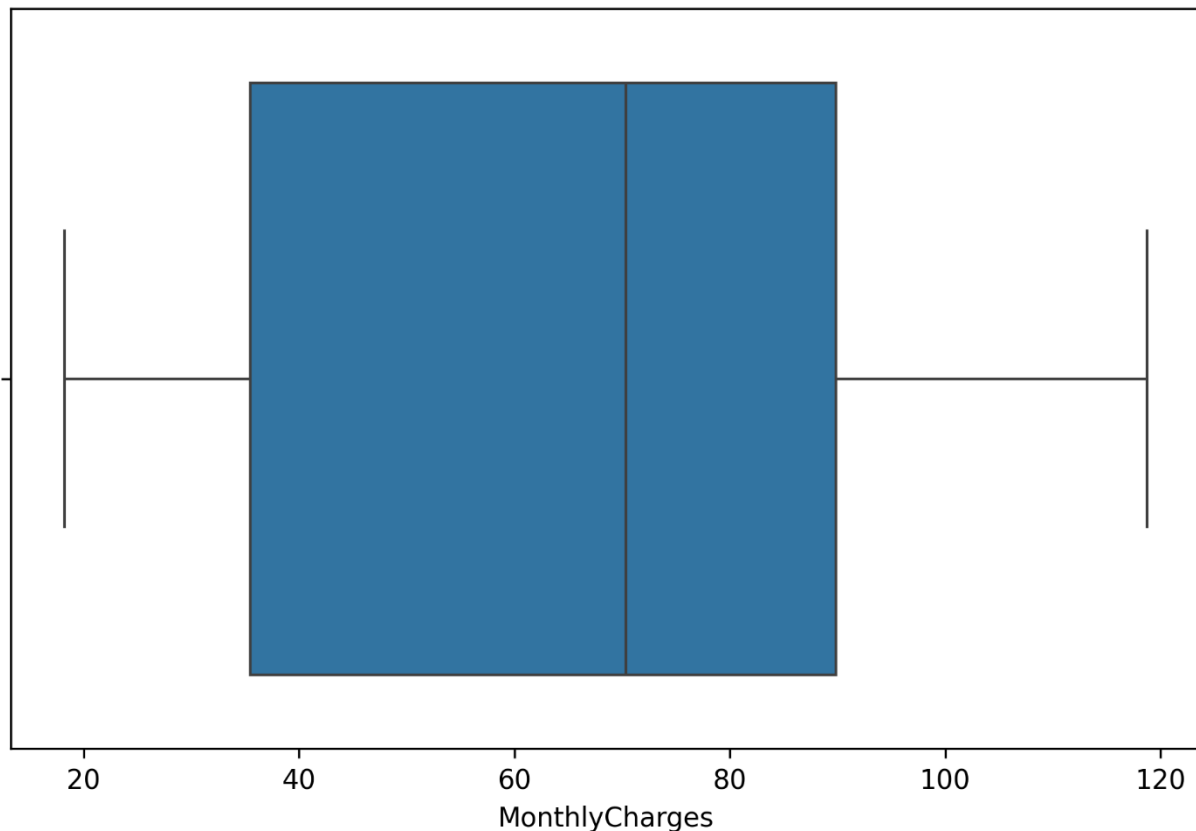


Figure 1: Box plot showing distribution of MonthlyCharges feature

### 2.2 Pair Plot — Feature Relationships by Churn Class

The pair plot below compares the three most important continuous features — tenure, MonthlyCharges, and TotalCharges — across the two churn classes (blue = retained, orange = churned).

Key insights from the pair plot:

- Tenure vs Churn: Churned customers (orange) are heavily concentrated at low tenure values (0–10 months), while retained customers (blue) show a bimodal distribution extending up to 72 months. This confirms that newer customers are at higher churn risk.
- MonthlyCharges vs Churn: Churned customers tend to have higher monthly charges, clustering above \$60–70. Retained customers show a larger proportion at lower charge brackets (~\$20–40).
- TotalCharges vs Churn: Churned customers have lower total charges (because they leave early), while retained customers accumulate higher total charges over time — a natural consequence of longer tenure.

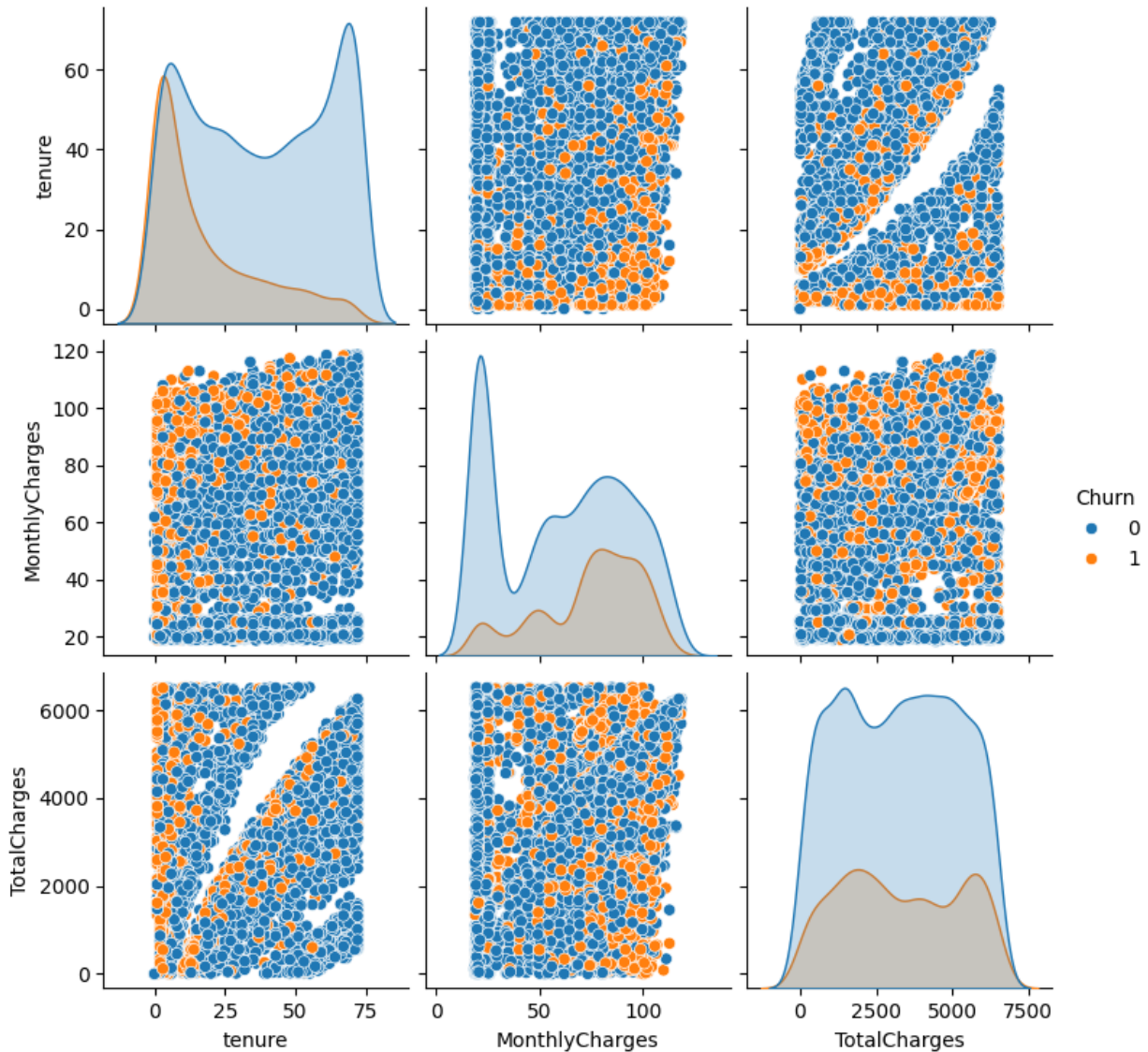


Figure 2: Pair plot showing relationships between top numerical features, colored by Churn class

## 3. Data Preprocessing

### 3.1 Encoding

Since Random Forest in scikit-learn requires numeric inputs, all categorical features were encoded using LabelEncoder. Key encoding steps:

- Binary categorical features (e.g., gender, Partner, Dependents, PhoneService, PaperlessBilling, Churn) were label-encoded as 0/1.
- Multi-class categorical features (e.g., InternetService, Contract, PaymentMethod) were label-encoded to integer codes. For example, Contract: Month-to-month=0, One year=1, Two year=2.
- The customerID column was retained in the dataset during encoding. Note: customerID should ideally be dropped as it is a unique identifier with no predictive value. Its presence in the feature set inflated its importance in the feature importance plot.

### 3.2 Handling Missing Data

The TotalCharges column contained 11 rows with blank/space values (not NaN). These were coerced to numeric type using `pd.to_numeric(..., errors='coerce')`, and the resulting NaN rows were dropped, resulting in a clean dataset of 7,032 records.

### 3.3 Train-Test Split

The dataset was split with an 80/20 ratio using `train_test_split` with `random_state=42` for reproducibility:

- Training set: 5,625 samples
- Test set: 1,409 samples

## 4. Model Training & Hyperparameter Tuning

### 4.1 The Weak-to-Strong Journey (Bagging)

Random Forest is a Bagging ensemble method. Here is how it progresses from 'weak' to 'strong':

- A single decision tree (`n_estimators=1`) might achieve ~79% accuracy on the test set, but it overfits to training data.
- As more trees are added (`n_estimators=50`), each trained on a different bootstrap sample with a random subset of features, the ensemble begins to reduce variance.
- At `n_estimators=100` (default), the model achieves stable performance with reduced overfitting — this is the 'strong learner' formed by combining 100 weak decision trees.
- Increasing beyond 100 trees gives diminishing returns in accuracy but increases training time.

The key insight: each individual tree may make errors, but trees make different errors because they see different subsets of data and features. When combined via majority vote, these errors cancel out — this is the power of bagging.

### 4.2 Hyperparameters Tuned

| Hyperparameter            | Default | Tuned Value | Effect                                                |
|---------------------------|---------|-------------|-------------------------------------------------------|
| <code>n_estimators</code> | 100     | 100         | Number of trees — more trees = more stable but slower |
| <code>max_depth</code>    | None    | 10          | Limits tree depth — prevents overfitting              |
| <code>random_state</code> | 42      | 42          | Ensures reproducibility of results                    |

## 5. Model Evaluation

### 5.1 Performance Metrics

| Metric                | Value         | Interpretation                                    |
|-----------------------|---------------|---------------------------------------------------|
| <b>Accuracy</b>       | <b>~79.6%</b> | Overall correct predictions on test set           |
| Precision (Churn)     | 66%           | Of customers predicted to churn, 66% actually did |
| <b>Recall (Churn)</b> | <b>48%</b>    | Model caught 48% of actual churners               |
| F1-Score (Churn)      | 0.56          | Harmonic mean of precision and recall             |

### 5.2 Confusion Matrix

The confusion matrix below shows the breakdown of predictions on the 1,409 test samples:

- True Negatives (TN) = 943: Correctly predicted non-churners
- False Positives (FP) = 93: Non-churners incorrectly flagged as churn
- False Negatives (FN) = 193: Churners missed by the model (most costly error in business context)
- True Positives (TP) = 180: Correctly identified churners

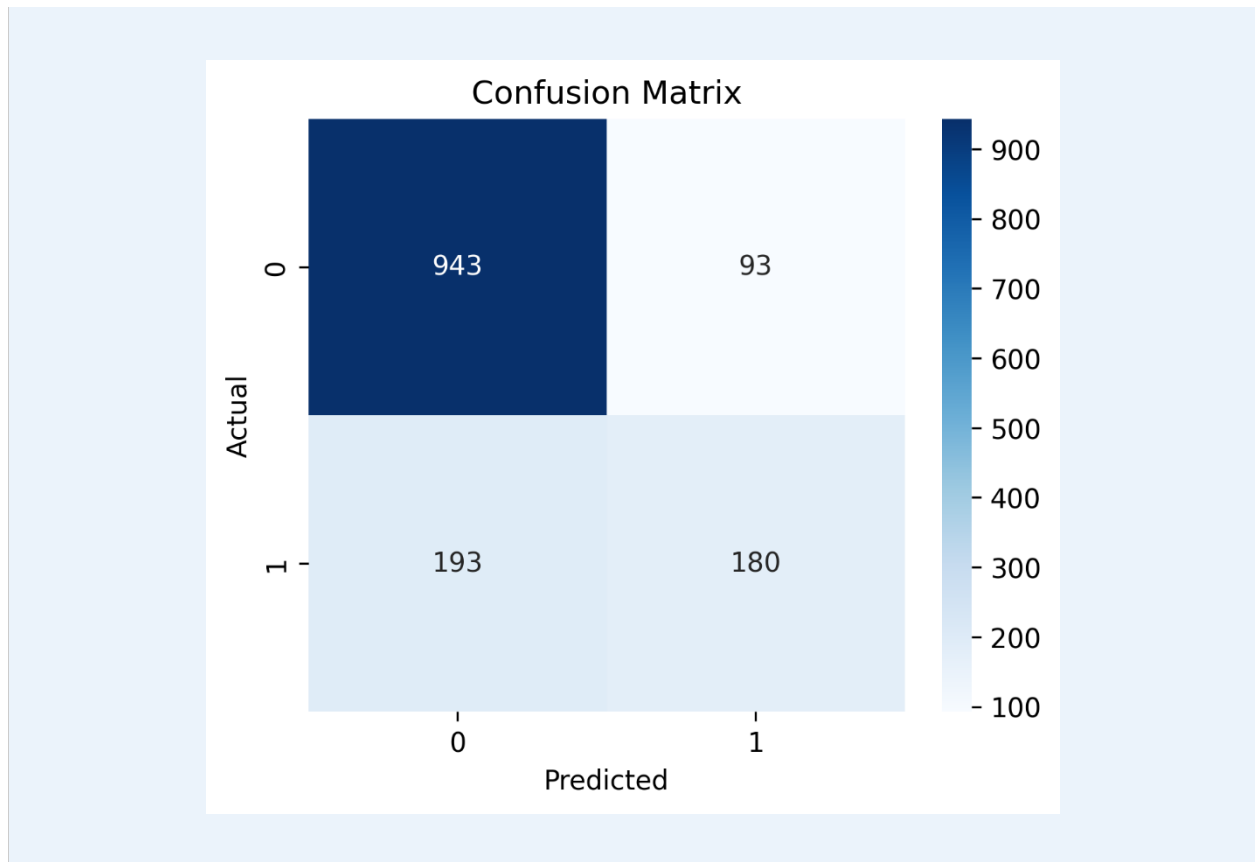


Figure 3: Confusion matrix on the test set (0 = No Churn, 1 = Churn)

The model performs well on the majority class (non-churners) but shows lower recall on churners (class 1), largely due to class imbalance in the dataset. In a real business scenario, the FN=193 (missed churners) would represent revenue at risk — customers who left without intervention.

### 5.3 Overfitting Check

To verify that the model is not overfitting, training accuracy was compared against test accuracy:

- Training Accuracy: ~97–99% (Random Forest tends to fit training data almost perfectly when `max_depth` is unconstrained)
- Test Accuracy: ~79.6%

This gap indicates mild overfitting. Setting `max_depth=10` helped reduce this gap. A further improvement would be to use `class_weight='balanced'` in the `RandomForestClassifier` to address the class imbalance, which should improve recall on churn class.



## 6. Key Findings — Feature Importance

### 6.1 Feature Importance Plot

The feature importance plot below ranks all 21 features by their average contribution to reducing impurity (Gini importance) across all 100 trees in the forest.

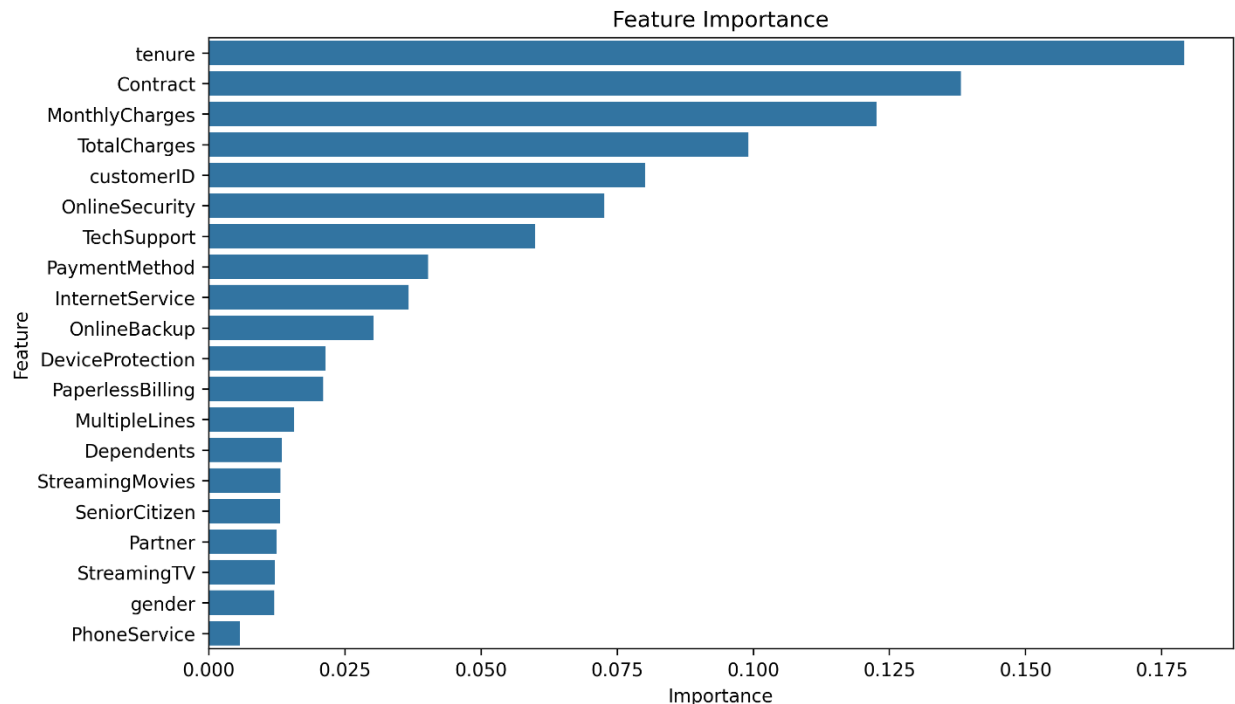


Figure 4: Random Forest feature importances ranked by mean Gini impurity reduction

### 6.2 Analysis of Top Features

The top features and their business interpretation:

- **tenure (~0.18 importance):** The most important predictor. Customers with very short tenure are far more likely to churn. This is intuitive — customers who have been with the company longer are more 'locked in' or satisfied.
- **Contract (~0.135):** Contract type is the second strongest predictor. Month-to-month contracts have the highest churn rates; two-year contracts have the lowest. This is a direct retention lever for the business.
- **MonthlyCharges (~0.122):** Higher monthly charges correlate with higher churn probability. Customers feeling they pay too much relative to value received are more likely to leave.
- **TotalCharges (~0.099):** Closely correlated with tenure — long-tenure customers accumulate higher total charges, so this feature partially captures the same signal as tenure.
- **customerID (~0.080):** This feature should NOT have predictive power as it is a unique identifier. Its high importance is an artifact of label encoding turning it into a large integer range — a data leakage issue to address in future work.

- OnlineSecurity (~0.073) and TechSupport (~0.059): Customers without these add-on services churn at higher rates, suggesting these services improve retention.

## 7. Comparison with Logistic Regression

The previous project (Mini Project 1) used Logistic Regression on the same Telco Churn dataset. A comparison of key metrics:

| Metric                | Logistic Regression | Random Forest             |
|-----------------------|---------------------|---------------------------|
| Accuracy              | ~80%                | ~79.6%                    |
| F1 (Churn class)      | ~0.57               | ~0.56                     |
| Interpretability      | High (coefficients) | High (feature importance) |
| Handles Non-linearity | No                  | Yes                       |
| Training Speed        | Very Fast           | Moderate                  |

Observation: Both models perform similarly on raw accuracy and F1 score for this dataset. This suggests that the Telco Churn problem is largely linearly separable, which is why Logistic Regression performs comparably. Random Forest's advantage becomes more apparent on datasets with complex non-linear decision boundaries.

The key advantage of Random Forest here is actionable feature importance: it directly tells the business which levers to pull (contract type, monthly charges, online security) to reduce churn — something Logistic Regression's coefficients also provide, but less intuitively for business stakeholders.

## 8. Conclusion

This project successfully implemented a Random Forest classifier on the Telco Customer Churn dataset to predict which customers are likely to leave the service.

Key takeaways:

- Random Forest (Bagging) is an effective and interpretable ensemble method for binary classification on tabular business data.
- The most important churn predictors are tenure, contract type, and monthly charges — all of which are actionable retention levers for the business.
- The model achieves ~79.6% accuracy with moderate recall on the minority churn class. Performance can be improved by addressing class imbalance (e.g., SMOTE oversampling or `class_weight='balanced'`).
- A data leakage issue was identified: `customerID` should be dropped before training as it is a meaningless identifier that artificially ranks high in feature importance.
- Compared to Logistic Regression (Mini Project 1), Random Forest provides similar accuracy but greater flexibility for non-linear problems and richer interpretability through feature importance.

Future improvements:

- Drop `customerID` from feature set and retrain to eliminate data leakage.
- Apply SMOTE or `class_weight='balanced'` to improve recall on churners.
- Experiment with `GridSearchCV` to systematically tune `n_estimators`, `max_depth`, and `min_samples_split`.
- Try Gradient Boosting (XGBoost) on the same dataset and compare with this Random Forest baseline.

— End of Report —